



HTML5

Session2

Microsoft
Be what's next.™

Microsoft® Student Partners





Semantic Elements

- Semantic = Meaning.
- Semantic elements = Elements with meaning.
- A semantic element clearly describes its meaning to both the browser and the developer.
- **non-semantic** elements: `<div>` and `<span>` - Tells nothing about its content.
- **semantic** elements: `<form>`, `<table>`, and `<img>` - Clearly defines its content.



New Semantic Elements

- Many of existing web sites today contains HTML code like this:

```
<div id="nav">
```

```
<div class="header">, or
```

```
<div id="footer">
```

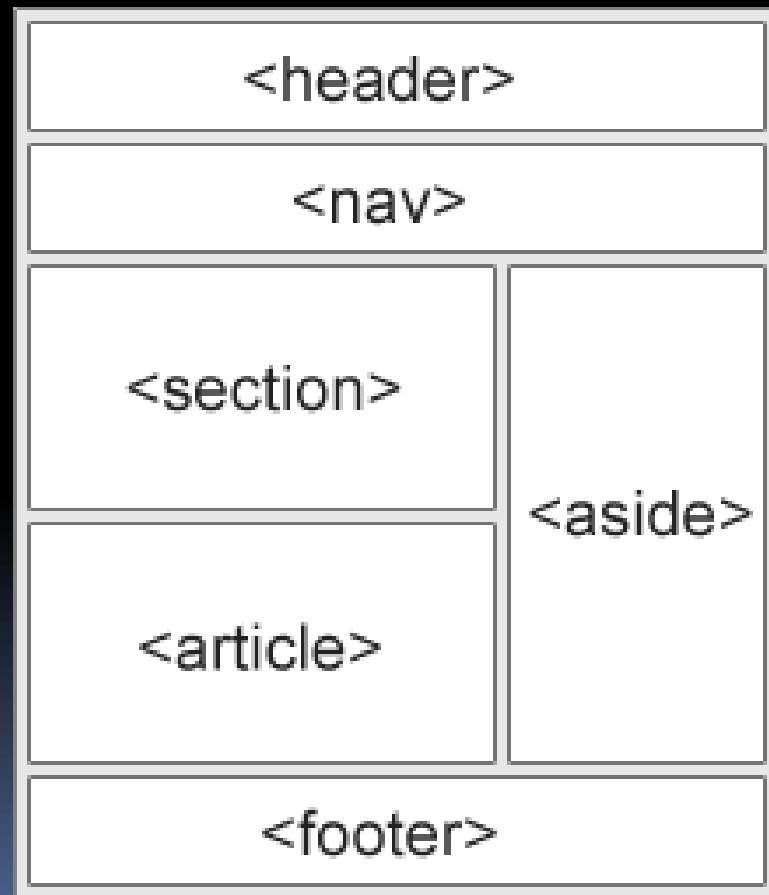
to indicate navigation links, header, and footer.

Semantic Elements...



- HTML5 offers new semantic elements to clearly define different parts of a web page.

- <header>
- <nav>
- <section>
- <article>
- <aside>
- <figure>
- <figcaption>
- <footer>
- <details>
- <summary>
- <mark>
- <time>



HTML5 <section> Element



- The <section> element defines a section in a document.
- According to W3C's HTML5 documentation: "A section is a thematic grouping of content, typically with a heading."
- The <section> tag defines sections in a document. Such as chapters, headers, footers, or any other sections of the document.

```
<section>
<h1>WWF</h1>
<p>The World Wide Fund for Nature (WWF) is....</p>
</section>
```

HTML5 <article> Element



- The <article> element specifies independent, self-contained content.
- An article should make sense on its own and it should be possible to distribute it independently from the rest of the web site.
- Examples of where an <article> element can be used:
 - Forum post
 - Blog post
 - News story
 - Comment

```
<article>
```

```
<h1>Internet Explorer 9</h1>
```

```
<p>Windows Internet Explorer 9  
(abbreviated as IE9) was released to  
the public on March 14, 2011 at 21:00  
PDT.....</p>
```

```
</article>
```



HTML5 <nav> Element

- The <nav> element defines a set of navigation links.
- The <nav> element is intended for large blocks of navigation links.
- However, not all links in a document should be inside a <nav> element!

```
<nav>
<a href="/html/">HTML</a> |
<a href="/css/">CSS</a> |
<a href="/js/">JavaScript</a> |
<a href="/jquery/">jQuery</a>
</nav>
```



HTML5 <aside> Element

- The <aside> element defines some content aside from the content it is placed in (like a sidebar).
- The aside content should be related to the surrounding content.

```
<p>My family and I visited The Epcot center this summer.</p>  
<aside>  
  <h4>Epcot Center</h4>  
  <p>The Epcot Center is a theme park in Disney World,  
  Florida.</p>  
</aside>
```




HTML5 <header> Element

- The <header> element specifies a header for a document or section.
- The <header> element should be used as a container for introductory content.
- You can have several <header> elements in one document.

```
<article>
<header>
  <h1>Internet Explorer 9</h1>
  <p><time pubdate datetime="2011-03-15"></time></p>
</header>
<p>Windows Internet Explorer 9 (abbreviated as IE9) was released to
the public on March 14, 2011 at 21:00 PDT.....</p>
</article>
```

HTML5 <footer> Element



- The <footer> element specifies a footer for a document or section.
- A <footer> element should contain information about its containing element.
- A footer typically contains the author of the document, copyright information, links to terms of use, contact information, etc.
- You can have several <footer> elements in one document.

```
<footer>
<p>Posted by: Hege Refsnes</p>
<p><time pubdate datetime="2012-03-
01"></time></p>
</footer>
```



HTML5 `<figure>` and `<figcaption>` Elements

- The `<figure>` tag specifies self-contained content, like illustrations, diagrams, photos, code listings, etc.
- While the content of the `<figure>` element is related to the main flow, its position is independent of the main flow, and if removed it should not affect the flow of the document.
- The `<figcaption>` tag defines a caption for a `<figure>` element.
- The `<figcaption>` element can be placed as the first or last child of the `<figure>` element.

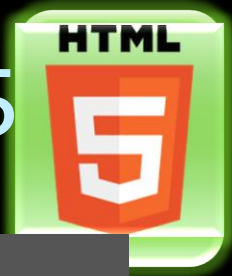
<figure> and <figcaption> Elements Example



Brighten your day with an exquisite cup of coffee

```
<figure>
  
  <figcaption>Brighten your day with an exquisite cup of
coffee.</figcaption>
</figure>
```

Semantic Elements in HTML5



Tag	Description
<u><article></u>	Defines an article
<u><aside></u>	Defines content aside from the page content
<u><details></u>	Defines additional details that the user can view or hide
<u><figcaption></u>	Defines a caption for a <figure> element
<u><figure></u>	Specifies self-contained content, like illustrations, diagrams, photos, code listings, etc.
<u><footer></u>	Defines a footer for a document or section
<u><header></u>	Specifies a header for a document or section
<u><main></u>	Specifies the main content of a document
<u><mark></u>	Defines marked/highlighted text
<u><nav></u>	Defines navigation links
<u><section></u>	Defines a section in a document
<u><summary></u>	Defines a visible heading for a <details> element
<u><time></u>	Defines a date/time



HTML5 Form Elements

- HTML5 has the following new form elements:
- `<datalist>`
- `<keygen>`
- `<output>`
- `<date>`
- `<datetime>`
- `<url>`
- `<email>`
- `<tel>`
- `<color>`

A screenshot of a 'Create an Account' form. The form has a dark gray background with white text. It contains four input fields: 'Name' (with a person icon), 'Phone Number' (with a phone icon), 'Email' (with an envelope icon), and 'Password' (with a lock icon). Below the fields is a checkbox labeled 'I agree to the Terms and Conditions and Privacy Policy'. At the bottom right is a green 'Create Account' button.

Not all browsers support all the new form elements. However, you can already start using them; If they are not supported, they will behave as regular text fields.

HTML5 <datalist> Element



- The <datalist> element specifies a list of pre-defined options for an <input> element.
- The <datalist> element is used to provide an "autocomplete" feature on <input> elements. Users will see a drop-down list of pre-defined options as they input data.
- Use the <input> element's list attribute to bind it together with a <datalist> element.



<datalist> Element Example

- An <input> element with pre-defined values in a <datalist>:

```
<input list="browsers">  
<datalist id="browsers">  
  <option value="Internet Explorer">  
  <option value="Firefox">  
  <option value="Chrome">  
  <option value="Opera">  
  <option value="Safari">  
</datalist>
```

A screenshot of a web form. On the left is a dropdown menu with a blue border and a downward arrow. The menu is open, showing a list of browser names: Firefox, Internet Explorer, Firefox, Chrome, Opera, and Safari. The first 'Firefox' is selected. To the right of the dropdown is a grey button with the text 'Submit' in black.



HTML5 <keygen> Element

- The purpose of the <keygen> element is to provide a secure way to authenticate users.
- The <keygen> tag specifies a key-pair generator field in a form.
- When the form is submitted, two keys are generated, one private and one public.
- The private key is stored locally, and the public key is sent to the server.
- The public key could be used to generate a client certificate to authenticate the user in the future.



<keygen> Element Example

- A form with a keygen field:

```
<form action="demo_keygen.asp" method="get">  
Username: <input type="text" name="usr_name">  
Encryption: <keygen name="security">  
<input type="submit">  
</form>
```

Username: Encryption:

HTML5 <output> Element



- The <output> element represents the result of a calculation (like one performed by a script).
- Perform a calculation and show the result in an <output> element:

```
<form  
oninput="x.value=parseInt(a.value)+parseInt(b.value)">  
<input type="range" id="a" value="50">100 +  
<input type="number" id="b" value="50">=  
<output name="x" for="a b"></output>  
</form>
```

0 100 + = 107



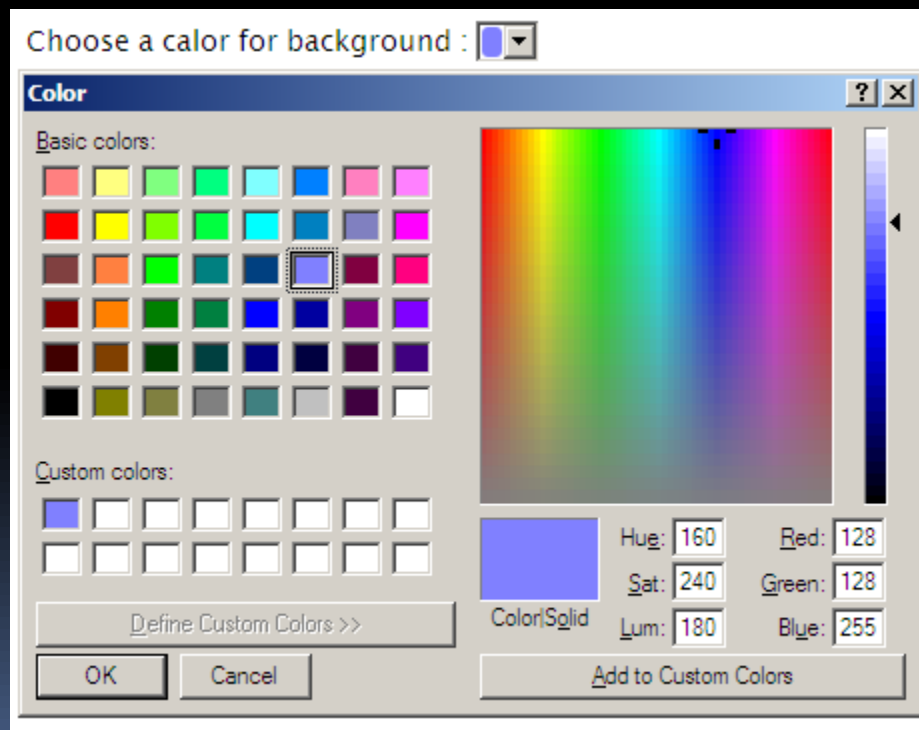
`<input type="color">`

- The `<input type="color">` defines a color picker.
- The default value is `#000000` (black).
- The value must be in seven-character hexadecimal notation.
- With Color input type, you no longer need a complex Javascript color picker, a simple line of code below will do the work.



`<input type="color">`

- `<input type="color" id="background-color" />`





`<input type="date">`

- The `<input type="date">` defines a date picker.
- The resulting value includes the year, month, and day.

Birthday: `<input type="date" name="bday">`

- A Date and time field can be easily found in many web forms.
- Typical applications are like ticket booking, appointment booking, ordering pizza and etc.
- The most commonly used solution for date input is to use Javascript date picker.



Date (<input type="date"/>)

- Apparently this is browser native date picker. You can only pick a specific date from the calendar.

Meeting Date :

January							2011
Mon	Tue	Wed	Thu	Fri	Sat	Sun	
27	28	29	30	31	1	2	
3	4	5	6	7	8	9	
10	11	12	13	14	15	16	
17	18	19	20	21	22	23	
24	25	26	27	28	29	30	
31	1	2	3	4	5	6	
Today							



Local Date and Time

- **`<input type="datetime-local"/>`**
- In compared to UTC time, you can have input date time value represents local time too.

A screenshot of a web form showing a date and time input field. The date is set to 2011-01-19 and the time is 11:50. A red circle highlights the time field. Below the input field is a calendar widget for January 2011. The calendar shows days of the week (Mon to Sun) and dates. The date 19 is highlighted in grey. A 'Today' button is at the bottom right of the calendar.

2011-01-19		11:50				
January 2011						
Mon	Tue	Wed	Thu	Fri	Sat	Sun
27	28	29	30	31	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

Today



Input Type : Email

- The `<input type="email">` defines a field for an e-mail address.
- The input value is automatically **validated** to ensure it is a properly formatted e-mail address.
- To define an e-mail field that allows multiple e-mail addresses, add the "multiple" attribute.

E-mail: `<input type="email" name="emailaddress">`



Input Type: url

- The `<input type="url">` defines a field for entering a URL.
- The input value is automatically validated before the form can be submitted.

Add your homepage:

`<input type="url" name="homepage">`

■



Input type : tel

- The `<input type="tel">` defines a field for entering a telephone number.

`<input type="tel" id="phone" name="phone" pattern="[0-9]{3}-[0-9]{3}-[0-9]{4}"`

`<small>Format: 123-456-7890</small>`

- Pattern - A regular expression the entered value must match to pass constraint validation



`<input>` autofocus Attribute

- The autofocus attribute is a boolean attribute.
- When present, it specifies that an `<input>` element should automatically get focus when the page loads.

First name:

```
<input type="text" name="fname" autofocus>
```



`<input>` required Attribute

- The required attribute is a boolean attribute.
- When present, it specifies that an input field must be filled out before submitting the form.
- The required attribute works with the following input types: text, search, url, tel, email, password, date pickers, number, checkbox, radio, and file.

Username: `<input type="text" name="usrname" required>`



HTML <iframe> Tag

- The <iframe> tag specifies an inline frame.
- An inline frame is used to embed another document within the current HTML document.

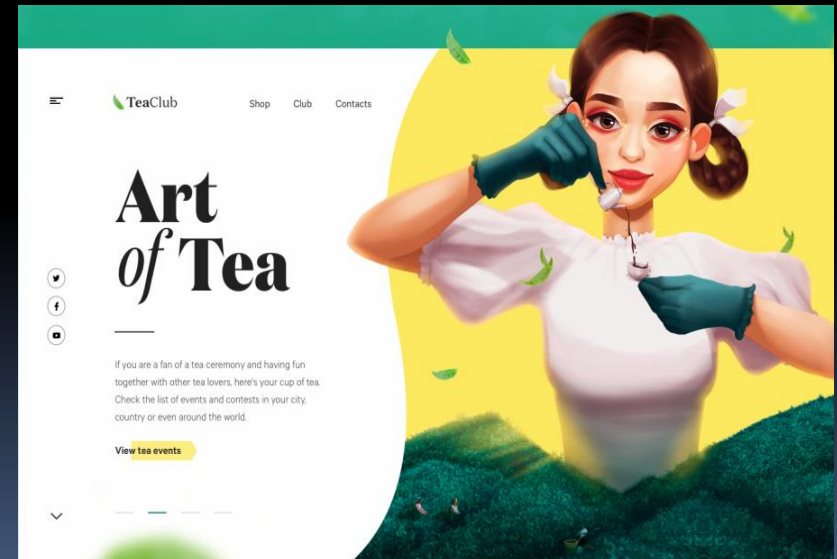
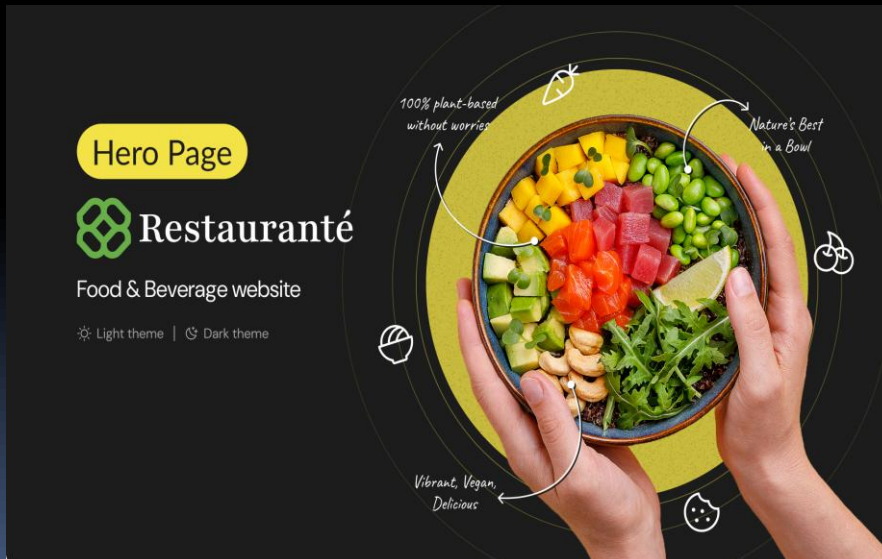
```
<iframe src="https://www.rediff.com">  
</iframe>
```

```
<iframe width="420" height="315"  
src="https://www.youtube.com/embed/tgbN  
ymZ7vqY?autoplay=1">  
</iframe>
```



Hero Image

- A Hero Image is a large image with text, often placed at the top of a webpage.
- Hero images can be designed using HTML and CSS.





Approach- To create Hero Image

- Create a container with a blurred background image using the ``.gfg`` class.
- Style the title, tagline, and button in the ``.logo`` class with border, padding, and positioning.
- Add a hover effect to the title for a gradient background.
- Design the button with background and text color, and a hover effect for interactivity.

HTML5 Media Elements



HTML
Video



HTML
Audio





HTML5 Video



- HTML5 provides a standard way for showing videos in modern websites.
- Before HTML5, there was no standard for showing videos/movies on web pages.
- Before HTML5, videos could only be played with a plug-in (like flash). However, different browsers supported different plug-ins.
- HTML5 defines a new element which specifies a standard way to embed a video or movie on a web page: the `<video>` element.



HTML5 Video - How It Works

```
<video width="320" height="240" controls>  
<source src="movie.mp4" type="video/mp4">  
<source src="movie.ogg" type="video/ogg">  
Your browser does not support the video tag.  
</video>
```



HTML5 Video - How It Works



- The control attribute adds video controls, like play, pause, and volume.
- It is also a good idea to always include width and height attributes.
- If height and width are set, the space required for the video is reserved when the page is loaded.
- However, without these attributes, the browser does not know the size of the video, and cannot reserve the appropriate space to it.
- The effect will be that the page layout will change during loading (while the video loads).
- You should also insert text content between the `<video>` and `</video>` tags for browsers that do not support the `<video>` element.
- The `<video>` element allows multiple `<source>` elements. `<source>` elements can link to different video files. The browser will use the first recognized format.

HTML5 Audio



- HTML5 provides a standard for playing audio files.
- Before HTML5, there was no standard for playing audio files on a web page.
- Before HTML5, audio files had to be played with a plug-in (like flash).
- However, different browsers supported different plug-ins.
- HTML5 defines a new element which specifies a standard way to embed an audio file on a web page: the <audio> element.

HTML5 Audio - How It Works



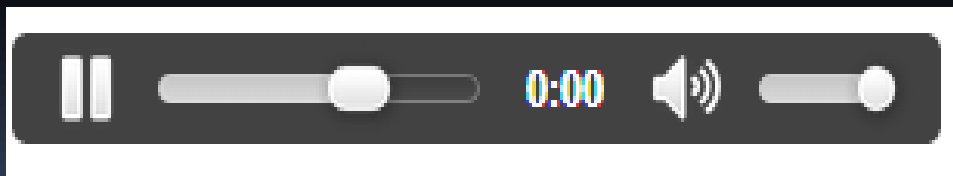
<audio controls>

<source src="horse.ogg" type="audio/ogg">

<source src="horse.mp3" type="audio/mpeg">

Your browser does not support the audio element.

</audio>



HTML Graphics



HTML5
< canvas >



HTML5 Canvas

- The HTML5 `<canvas>` element is used to draw graphics, on the fly, via scripting (usually JavaScript).
- The `<canvas>` element is only a container for graphics.
- You must use a script to actually draw the graphics.
- Canvas has several methods for drawing paths, boxes, circles, text, and adding images.



Create a Canvas

- A canvas is a rectangular area on an HTML page, and it is specified with the `<canvas>` element.
- By default, the `<canvas>` element has no border and no content.



```
<canvas id="myCanvas"  
width="200" height="100"  
style="border:1px solid #000000;">  
</canvas>
```

Draw Onto The Canvas With JavaScript



- First, find the <canvas> element.
- Then, call its getContext() method.
- The getContext("2d") object is a built-in HTML5 object, with many properties and methods for drawing paths, boxes, circles, text, images, and more.
- The fillStyle property can be a CSS color, a gradient, or a pattern.

```
<script>  
var c = document.getElementById("myCanvas");  
var ctx = c.getContext("2d");  
ctx.fillStyle = "#FF0000";  
ctx.fillRect(0,0,150,75);  
</script>
```





Canvas - Paths

- To draw straight lines on a canvas, we will use the following two methods:
 - **moveTo(x,y)** defines the starting point of the line
 - **lineTo(x,y)** defines the ending point of the line
- To actually draw the line, we must use one of the "ink" methods, like stroke().
- To draw a circle on a canvas, we will use the following method:
 - **arc(x,y,r,start,stop)**





Canvas - Text

- To draw text on a canvas, the most important property and methods are:
- font - defines the font properties for text
- fillText(*text*,*x*,*y*) - Draws "filled" text on the canvas
- strokeText(*text*,*x*,*y*) - Draws text on the canvas (no fill)

A white rectangular box with a thin black border, containing the text 'Hello World' in a simple, black, sans-serif font.

Hello World



Canvas - Gradients

- Gradients can be used to fill rectangles, circles, lines, text, etc.
- Shapes on the canvas are not limited to solid colors.
- There are two different types of gradients:
- `createLinearGradient(x,y,x1,y1)` - Creates a linear gradient
- `createRadialGradient(x,y,r,x1,y1,r1)` - Creates a radial/circular gradient



Gradients...

- Once we have a gradient object, we must add two or more color stops.
- The `addColorStop()` method specifies the color stops, and its position along the gradient.
- Gradient positions can be anywhere between 0 to 1.
- To use the gradient, set the `fillStyle` or `strokeStyle` property to the gradient, and then draw the shape, like a rectangle, text, or a line.

Gradients...



- Create a linear gradient. Fill rectangle with the gradient: Using `createLinearGradient()`



- Create a radial/circular gradient. Fill rectangle with the gradient: Using `createRadialGradient()`



HTML5 Inline SVG



- SVG stands for Scalable Vector Graphics
- SVG is used to define vector-based graphics for the Web
- SVG defines the graphics in XML format
- SVG graphics do NOT lose any quality if they are zoomed or resized
- Every element and every attribute in SVG files can be animated
- SVG is a W3C recommendation



SVG Advantages

- SVG images can be created and edited with any text editor
- SVG images can be searched, indexed, scripted, and compressed
- SVG images are scalable
- SVG images can be printed with high quality at any resolution
- SVG images are zoomable (and the image can be zoomed without degradation)

Comparison of Canvas and SVG



Canvas	SVG
•Resolution dependent •No support for event handlers •Poor text rendering capabilities •You can save the resulting image as .png or .jpg •Well suited for graphic-intensive games	•Resolution independent •Support for event handlers •Best suited for applications with large rendering areas (Google Maps) •Slow rendering if complex (anything that uses the DOM a lot will be slow) •Not suited for game applications

HTML5 APIs



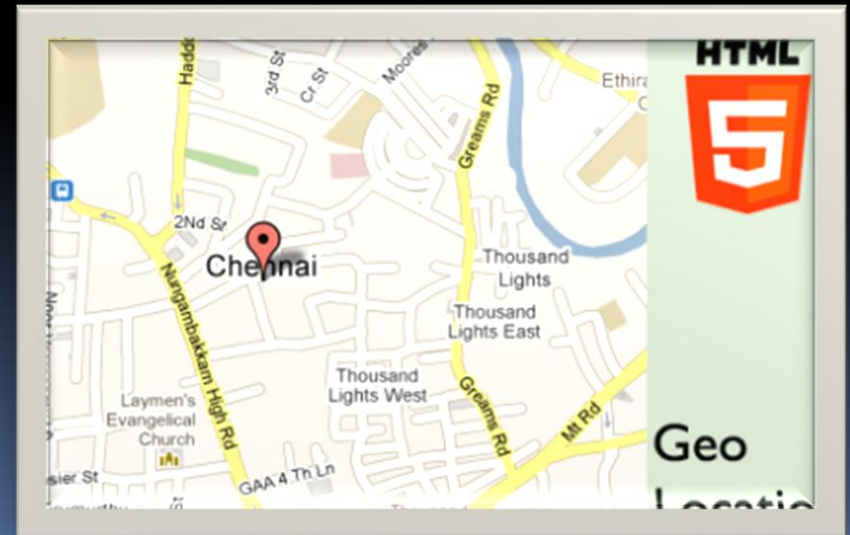
- Geolocation
- Drag/Drop
- Web Storage
- App Cache





HTML5 Geolocation

- The HTML5 Geolocation API is used to get the geographical position of a user.
- Since this can compromise user privacy, the position is not available unless the user approves it.
- Geolocation is much more accurate for devices with GPS, like iPhone.



Using Geolocation



- Use the `getCurrentPosition()` method to get the user's position.

```
<script>
var x = document.getElementById("demo");
function getLocation() {
    if (navigator.geolocation) {
        navigator.geolocation.getCurrentPosition(showPosition);
    } else {
        x.innerHTML = "Geolocation is not supported by this
browser.";
    }
}
function showPosition(position) {
    x.innerHTML = "Latitude: " + position.coords.latitude +
    "<br>Longitude: " + position.coords.longitude;
}
</script>
```



Drag and Drop

- Drag and drop is a very common feature.
-
- It is when you "grab" an object and drag it to a different location.
- In HTML5, drag and drop is part of the standard, and any element can be draggable.



Make an Element Draggable

- First of all: To make an element draggable, set the draggable attribute to true.



What to Drag - `ondragstart` and `setData()`

- Specify what should happen when the element is dragged.
- The **`ondragstart`** attribute calls a function, `drag(event)`, that specifies what data to be dragged.
- The **`dataTransfer.setData()`** method sets the data type and the value of the dragged data:

```
function drag(ev) {  
    ev.dataTransfer.setData("text", ev.target.id); }  
}
```

"text" and the value is the id of the draggable element ("drag1").



Where to Drop - ondragover

- The ondragover event specifies where the dragged data can be dropped.
- By default, data/elements cannot be dropped in other elements.
- To allow a drop, we must prevent the default handling of the element.
- This is done by calling the `event.preventDefault()` method for the ondragover event:

`event.preventDefault()`



Do the Drop - ondrop

- When the dragged data is dropped, a drop event occurs.
- The ondrop attribute calls a function, `drop(event)`:



Web Storage – Local & Session

- HTML local storage, better than cookies.
- With local storage, web applications can store data locally within the user's browser.
- Before HTML5, application data had to be stored in cookies, included in every server request.
- Local storage is more secure, and large amounts of data can be stored locally, without affecting website performance.
- Unlike cookies, the storage limit is far larger (at least 5MB) and information is never transferred to the server.
- Local storage is per domain. All pages, from one domain, can store and access the same data.

HTML Local Storage Objects



- HTML local storage provides two objects for storing data on the client:

`window.localStorage` - stores data with no expiration date

`window.sessionStorage` - stores data for one session (data is lost when the browser tab is closed)



The localStorage Object

- The localStorage object stores the data with no expiration date.
- The data will not be deleted when the browser is closed, and will be available the next day, week, or year.

// Store

```
localStorage.setItem("lastname", "Smith");
```

// Retrieve

```
document.getElementById("result").innerHTML  
= localStorage.getItem("lastname");
```

- localStorage name/value pair

The sessionStorage Object



- The sessionStorage object is equal to the localStorage object, **except** that it stores the data for only one session.
- The data is deleted when the user closes the specific browser tab.

MathML



- It is used to represent the mathematical notations in web pages and which is supported by all mathematical symbols.
- It is known as **Mathematical Markup Language** and is used to demonstrate some mathematical notations.
- **W₃C**(World Wide Web Consortium) will maintain all the standards of MathML.
- In order to get the Mathematical signs, Notations, symbols into the HTML5 document user need to use the MathML elements mainly it is focus on structure and content.



MathML - Advantages

- MathML is flexible to convert from one form to another form and generate the several form of output like text displays, graphical displays, speech synthesizers, and print devices which encodes the mathematical notations taking the structure and content of the document.
- MathML is very flexible which allows to pass the information from several types of documents and renders.
- Encoded data can be used by any of the software to process or generate the data.



MathML - Specifications

MathML can be divided into 2 types

- **Presentation Markup**
Is used to mark out the Layout and Mathematical expression.
- **Content Markup**
Is used to mark up the content of the document.



MathML presentation elements

- `<math>` (Top-level element)
- `<mi>` (Identifier)
- `<mn>` (Number)
- `<mo>` (Operator)
- `<ms>` (String literal)
- `<msub>` (Subscript)
- `<msup>` (Superscript)
- `<mtable>` (Table or matrix)
- `<mrow>` (Grouped sub-expressions)

MathML - Example



```
<!doctype html>
<head>  <meta charset="UTF-8">
        <title>Pythagorean theorem</title> </head>
<body>
  <math xmlns="http://www.w3.org/1998/Math/MathML">
    <mrow>
      <msup><mi>a</mi><mn>2</mn></msup>
      <mo>+</mo>
      <msup><mi>b</mi><mn>2</mn></msup>
      <mo>=</mo>
      <msup><mi>c</mi><mn>2</mn></msup>
    </mrow>
  </math>  </body> </html>
```



Server Sent Events

- It is used to generate the events from web applications to the web browsers.
- The events that are flowing from web browser to the web server are known as the Server Sent Events.

Ex: User clicking a link to generate the next page.
In order to push the DOM events continuously from web server to the browser use SSE.



Server Sent Events

- SSE creates a persistent connection to the server when the new data is available then sends the data to the clients.
- Server Sent Events are flown from server to the client in the form of stream data.
- In order to use the Server Sent Events add the **<eventsource>** element to the document.
- After adding the element to the document, source attribute points to the URL provided, then HTTP connection sends the data with events.
- The URL points on a server side script that should be the PHP, PERL, or Python then it will send the data continuously.

Web SQL Database



- The Web SQL Database is that It is not actually part of the HTML5 specification.
- But it is part of the suite of specifications that allows us developers to build fully fledged web applications with client-side databases using SQL



How to use Web SQL Database

- There are following three core methods which you must know in order to work with Web SQL Database –
- **openDatabase** – It is used to create a the database object. If database is already created, than it will use it.
- **transaction** – It is used to performing either commit or roll-back a transaction. Which provide us the ability to control a transaction based on the situation.
- **executeSql** – It is used to execute a written SQL query. e.g- insert, delete any record.



openDatabase - Using a Database

- This method allow us to use a database if it already exists, and create the database if it does not exist.
- Following example code shows how to use this method (openDatabase) to create and open a database.
- **`var mydb = openDatabase('db_name', '1.0', 'Any Description', 2 * 1024 * 1024);`**



transaction - Executing a queries

- To execute a query we need to use the `database.transaction()` method.
- This method required only one argument, which itself a method that takes will execute the actual query.
- Following are the sample code to show how to use `database.transaction()`.

```
mydb.transaction(function (tx) {  
    tx.executeSql('CREATE TABLE IF NOT EXISTS  
HTMLDEMO (id unique, example)');  
});
```



INSERT Operation

- If you want to insert any record in above created table, than we need to use following simple SQL query.
- **tx.executeSql('INSERT INTO HTMLDEMO (id, example) VALUES (1, "Html 5 Web Storage")');**



SELECT Operation

- Select a record from the database we have created is very easy. we just need to use simple select query.

```
mydb.transaction(function (tx) {  
    tx.executeSql('SELECT * FROM HTMLDEMO', [], function  
(tx, results) {  
        var count = results.rows.length;  
        msg = "<p>Total rows: " + count + "</p>";  
        for (var i = 0; i < count; i++){  
            alert(results.rows.item(i).example);  
        }  
        }, null);  
});
```



END

